ResQGrid: Disaster Relief & Coordination Platform

Core Concept: A centralized, AI-driven dashboard and mobile-responsive web app that acts as a single source of truth for victims, volunteers, NGOs, and government agencies during a crisis.

Core Features to Build

- **Victim SOS & Triage:** A low-bandwidth form (or SMS gateway integration) for victims to ping their location and needs.
- **AI Priority Engine:** An algorithm that auto-categorizes and flags high-risk requests (e.g., medical emergencies, trapped individuals) over standard supply drops.
- **Location Intelligence Dashboard:** A live map for authorities showing heatmaps of SOS pings, volunteer live locations, and safe zone perimeters.
- **Resource & Shelter Tracker:** Real-time capacity counters for shelters and inventory management for NGOs to prevent supply duplication.

6-Person Team Division of Labor

Team Member	Role	Core Responsibilities for this Prototype
Member 1	Frontend Developer (Victim/Volunteer UI)	Build mobile-responsive screens for the SOS ping, volunteer registration, and task acceptance UI. Focus on a lightweight, low-bandwidth design.
Member 2	Frontend Developer (Admin/NGO Dashboard)	Develop the central command dashboard. Integrate map widgets, heatmaps, and data visualization for authorities to track real-time operations.
Member 3	Backend Developer (APIs & Database)	Set up the server and database architecture. Write APIs to handle SOS requests, update shelter capacities, and manage user roles (Victim, Admin, NGO).
Member 4	Location Intel & Backend Support	Handle the geolocation data logic. Implement spatial queries (e.g., finding the nearest volunteer to an SOS ping) and assist Member 3 with API integrations.
Member 5	Data & Security Engineer	Develop the priority-based resource allocation logic using Python to score and rank incoming requests. Build a secure logging framework to parse, encrypt, and monitor network requests across the application.
Member 6	Project Manager & Tech Writer	Create the system architecture diagrams, write the presentation report, test the platform for bugs, and

Team Member	Role	Core Responsibilities for this Prototype
		prepare the demo script.

Accelerated Action Plan (Aug 6 – Aug 14)

Date	Phase	Action Items
Aug 6	Architecture & Setup	Finalize tech stack. Setup GitHub repository. Member 6 creates UI wireframes; Member 3 & 4 design the database schema.
Aug 7	Core Infrastructure	Members 1 & 2 build static frontend screens. Member 3 builds basic CRUD APIs. Member 5 writes the initial priority algorithm logic.
Aug 8	Location & Mapping	Member 4 integrates maps and geolocation features into the backend. Members 1 & 2 connect their frontend screens to the live APIs.
Aug 9	Logic Integration	Link the AI priority algorithm (Member 5) to the SOS intake pipeline. Connect the volunteer tracking to the Admin map dashboard.
Aug 10	Feature Freeze	No new features. All members focus strictly on connecting any broken links between the frontend and backend.
Aug 11	QA, Bug Fixes & Logs	Complete platform testing. Member 5 reviews data logs and secures the app. Member 6 finalizes the technical report and generates demo data.
Aug 12	Review & Rehearsal	Development is halted. The entire team reviews the system architecture, learns the platform's data flow, and practices the presentation.
Aug 13	Polish & Pitch Prep	Conduct mock Q&A sessions. Finalize slide decks and ensure the live demo environment is perfectly stable.
Aug 14	**Delivery Day**	Present the ResQGrid platform and submit the final report.

Recommended Technology Stack

- **Frontend:** React.js or Flutter
- **Backend:** Python (Django/FastAPI) or Node.js
- **Database:** PostgreSQL with the PostGIS extension
- **Mapping:** Mapbox API or Google Maps Platform

Glossary of Technical Terms

Application & System Architecture

- **API (Application Programming Interface):** A set of rules allowing different software programs to communicate. Here, APIs allow the mobile app to send SOS requests to the database.
- **Backend:** The server, database, and logic that processes user requests, assigns priorities, and manages data.
- **CRUD Operations:** Create, Read, Update, and Delete. The four basic functions to manage data.
- **Database Schema:** The structural blueprint defining how data is organized and connected.
- **Frontend / UI:** The visual interface (buttons, maps, forms) users interact with.
- **MVP (Minimum Viable Product):** A version of the platform with only the most essential features required to demonstrate its value.
- **Tech Stack:** The combination of programming languages and frameworks used (e.g., React, Python, PostgreSQL).

Data & Geolocation

- **Geolocation:** Identifying the real-world geographic location of a device.
- **Heatmap:** Visual data representation on a map where value density is shown by color.
- **Location Intelligence:** Analyzing geographic data to connect victims with nearby resources.
- **PostGIS / Spatial Queries:** A database extension for spatial questions (e.g., "Find all volunteers within a 5-km radius").

Disaster Relief & Operations

- **Low-Bandwidth Optimization:** Designing the app to function on very slow or dropping internet connections.
- **Resource Allocation:** Strategic distribution of limited supplies and personnel.
- **Single Source of Truth:** A central system ensuring all agencies view the same real-time data.
- **Triage:** Prioritizing victims based on the severity of their needs.
- **Zero-Duplication:** Preventing multiple NGOs from sending the same supplies to one location.

Deliverables Checklist (To complete by August 12)

August 6: Architecture & Environment Setup

- [ ] **Frontend:** Initialize repositories; finalize wireframes for SOS interface and Dashboard.
- [ ] **Backend & Security:** Set up Python server, configure PostgreSQL, and implement baseline encryption for victim data.
- [ ] **Location Intel:** Obtain API keys; configure PostGIS spatial extension.
- [ ] **PM:** Draft Team Details, Problem Statement Selection, and Idea Summary for the SIH report.

August 7: Core Infrastructure & Logic

- [ ] **Frontend:** Complete static screens for SOS ping form and shelter capacity tables.
- [ ] **Backend:** Finalize core CRUD operations for SOS requests and volunteers.
- [ ] **Data & Security:** Write Python scripts for priority engine; establish secure logging framework.
- [ ] **PM:** Outline Technical Specifications and Methodology report sections.

August 8: Location Integration

- [ ] **Frontend:** Connect UI forms to backend APIs; render live map on Admin Dashboard.
- [ ] **Location Intel:** Write spatial queries linking victim coordinates with nearest volunteers/safe zones.
- [ ] **PM:** Draft Feasibility & Impact section of the report.

August 9: System Integration

- [ ] **Frontend:** Visually integrate AI priority scores into UI; add heatmaps to Dashboard map.
- [ ] **Backend & Security:** Connect priority algorithms and spatial queries to the SOS intake pipeline; verify end-to-end data transfer security.

August 10: Feature Freeze & Consolidation

- [ ] **All Devs:** Halt new features; connect broken endpoints and ensure smooth data flow.
- [ ] **PM:** Begin compiling final SIH deliverables.

August 11: QA, Testing & Final Fixes

- [ ] **Data & Security:** Final review of network logs to identify vulnerabilities/bottlenecks.
- [ ] **All Devs:** Patch critical bugs identified in testing.
- [ ] **PM:** Generate realistic demo data (simulated SOS pings, moving volunteers, shelter capacities).

August 12: Readiness Cutoff

- [ ] **Functional Prototype:** MVP is stable (handles SOS, priority sorting, map visualization without crashing).
- [ ] **Source Code:** Codebase is clean, commented, and committed to the main branch.
- [ ] **AI Logic:** Priority engine is fully integrated and successfully scores incoming data.
- [ ] **Documentation:** Final report is complete with architecture diagrams and cost-benefit analysis.
- [ ] **Pitch & Rehearsal:** Live demo environment is pre-loaded; team has practiced the presentation script and Q&A